C++ : gestion mémoire

TP C++

Cycle de vie et gestion mémoire : Réseau de métro

Eric Ramat
eric.ramat@univ-littoral.fr

6 octobre 2026

Durée : 3 heures

---

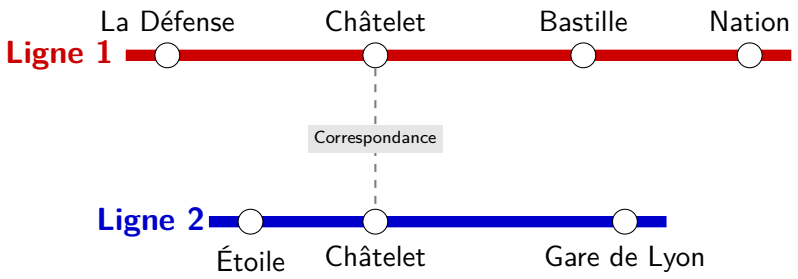
1 Objectifs

Dans cette première série de TP, vous allez construire progressivement une représentation d'un réseau de transport métropolitain (lignes, stations et correspondances) et, en chemin, explorer les concepts et les mécanismes suivants :

- la différence entre instances directes, références, pointeurs intelligents partagés (`shared_ptr`), pointeurs faibles (`weak_ptr`) et pointeurs uniques (`unique_ptr`),
- les avantages et limites de chaque approche,
- la gestion de la mémoire et des cycles de vie des structures de données.

Nous allons baser le code uniquement sur l'utilisation de structures (`struct`) sans classes C++. Le réseau de transport étudié est composé de deux lignes interconnectées :

- **Ligne 1 (Rouge)** :
  - Station 1 : La Défense
  - Station 2 : Châtelet (Station de correspondance)
  - Station 3 : Bastille
  - Station 4 : Nation
- **Ligne 2 (Bleue)** :
  - Station 1 : Étoile
  - Station 2 : Châtelet (Station de correspondance)
  - Station 3 : Gare de Lyon



2 Travail demandé

2.1 Avant de commencer ...

Avant de commencer le TP, il est nécessaire de forker le dépôt `git` contenant le projet avec un début de code. Ce code contient dans une arborescence : les fichiers pour cmake et un fichier main.cpp minimal. Vous devez voir apparaître dans votre compte gitlab (gitlab.dpt-info.univ-littoral.fr) le projet "metro". Après avoir forké le dépôt dans votre compte, à l'aide de la commande `git clone` sur votre machine, le code est disponible sur votre compte (en local). À ce stade, il est possible de vérifier que tout s'est bien passé en lançant la compilation à l'aide de la commande `cmake` :

```
$ mkdir build
$ cd build
$ cmake ..
$ make
$ ./src/metro
```

La réponse à chaque étape fera OBLIGATOIREMENT l'objet d'un commit dont le message sera de la forme "TP1 - Step x" et d'un commit pour les réponses aux questions de réflexion ("TP1 - Step x - Reflexion")

2.2 Rappel : compilation séparée

Afin d'optimiser la compilation, il faut créer deux fichiers par structure de données (`struct`) : un fichier d'en-têtes (.hpp) et un fichier contenant le corps des fonctions (.cpp).

Attention à bien protéger vos fichiers contre la double inclusion à l'aide du pragma once !

N'oubliez pas de modifier le fichier CMakeLists.txt du répertoire src afin qu'une modification dans un fichier `.hpp` ou `.cpp` n'engendre pas la compilation de la totalité du code.

2.3 Étape 1 — Instances directes

Dans une première version, vous représentez une station de métro par son nom (attribut `name`) ainsi que par la ligne à laquelle elle appartient (attribut `line_name`). Les connexions avec les stations adjacentes (attributs `next_station` et `previous_station`) sont stockées directement comme des **instances**. Cette première version est à définir dans un fichier `StationV1.hpp` et doit disposer d'un constructeur initialisant le nom de la station et de sa ligne.

L'affectation de la station suivante se fera par une affectation directe de l'attribut `next_station`.

Attention : stocker une station comme instance directe revient à en faire une **copie**. Une même station de correspondance (par exemple *Châtelet*) ne peut donc pas être **partagée** entre deux lignes différentes ou deux tronçons. Si la Ligne 1 et la Ligne 2 contiennent toutes deux la station *Châtelet*, chacune possédera sa **propre copie indépendante**, et non le même objet : une modification du nom ou de l'état sur l'une ne sera pas visible depuis l'autre. Gardez cette limite en tête, vous l'illustrerez avec du code.

Ajoutez une méthode `to_string` qui retourne une chaîne décrivant la station avec son nom, le nom de sa ligne, ainsi que les noms des stations suivante et précédente si elles existent. La méthode est à

déclarer dans le fichier .hpp et le corps de la méthode dans `StationV1.cpp`. Pour tester ce modèle, créez quelques exemples de stations liées entre elles (par exemple : *La Défense* suivie de *Châtelet*) dans le fichier `main.cpp`.

Complétez ensuite ce modèle en représentant, pour une structure `LineV1`, la liste de ses stations (attribut `stations`, sous forme d'un tableau ou d'un `std::vector` d'instances). Ajoutez une méthode `add_station` permettant d'ajouter une station à la ligne, et testez-la en ajoutant les stations de la Ligne 1.

Vous pouvez maintenant constater les limites de cette première approche : dès qu'il s'agit de représenter des stations **partagées** entre plusieurs lignes (correspondances) ou des liens bidirectionnels cohérents (suivante/précédente), la copie induite par les instances directes pose problème. C'est précisément ce que nous allons corriger dans l'étape suivante en utilisant des **références**.

Questions de réflexion à répondre dans le README et éventuellement dans le fichier main.cpp pour le code :

- Que se passe-t-il lorsque l'on modifie le nom de la station *Châtelet* sur la Ligne 1 via l'instance la représentant ? La modification apparaît-elle sur la Ligne 2 ? **Proposer le code**.
- Peut-on représenter une station de correspondance partagée entre deux structures de lignes sans dupliquer la mémoire ? **Proposer le code**.
- Peut-on représenter une station terminus sans station suivante (par exemple *Nation*) ?
- En ajoutant une station à une ligne via `add_station`, puis en modifiant l'instance originale hors du vecteur, la modification se répercute-t-elle dans le vecteur de la ligne ? Que cela vous apprend-il sur les instances directes ?

2.4 Étape 2 — Références

Dans une deuxième version, nous allons modifier la représentation des stations afin que les liens vers les stations adjacentes ou la ligne parente soient stockés comme des références plutôt que comme des instances. Le code est à écrire dans deux nouveaux fichiers : `StationV2.hpp` et `StationV2.cpp`. Vous devez proposer une nouvelle implémentation de la méthode `to_string`.

La représentation des listes de stations au sein d'une ligne n'est volontairement pas reprise ici sous forme de collection de références : comme vous le constaterez en répondant aux questions de réflexion, une collection de références (`std::vector<StationV2&>`) n'est pas réalisable en C++. Cette limitation sera levée à l'étape suivante grâce aux `shared_ptr`.

Utilisez le même exemple que l'étape 1 (Ligne 1 avec ses stations).

Questions de réflexion à répondre dans le README et éventuellement dans le fichier main.cpp pour le code :

- En quoi les références corrigent-elles certains problèmes de l'étape précédente ?
- Peut-on créer une station terminus sans station suivante sous forme de référence ? **Justifier votre réponse**.
- Peut-on stocker une collection de références (par ex. une liste de stations dans une ligne) ?
- Est-ce adapté pour un réseau métropolitain complet avec correspondances ? **Justifier votre réponse**.

2.5 Étape 3 — Pointeurs intelligents partagés (`shared_ptr`)

Dans une troisième version, nous allons utiliser les pointeurs intelligents partagés. Les liens entre stations et lignes sont désormais représentés par des `shared_ptr`. Deux nouveaux fichiers sont à définir : `StationV3.hpp` et `StationV3.cpp`. On profite de cette nouvelle structure pour ajouter la liste des stations dans la structure `LineV3` à l'aide de `std::vector<std::shared_ptr<StationV3»`.

Proposez la nouvelle implémentation de la méthode `to_string` qui affiche le nom de la station, le nom de sa ligne, la station suivante et la station précédente.

Ajoutez une méthode `add_station()` dans `LineV3` permettant d'ajouter une station à une ligne et de mettre à jour les liens bidirectionnels entre stations successives.

Afin de valider cette troisième version, on réalisera un test consistant à construire un petit réseau à l'aide de plusieurs objets `StationV3` reliés entre eux par des `shared_ptr`. On pourra créer la Ligne 1, puis y ajouter successivement *La Défense*, *Châtelet* et *Bastille*.
L'exécution du programme permettra ensuite d'afficher les informations complètes de chaque station grâce à la méthode `to_string()`, afin de vérifier que les liens amont et aval sont correctement établis dans les deux sens.
Voici la sortie console que vous devez obtenir :

```
La Defense - Line : Ligne 1, Next : Chatelet, Prev : terminus
Chatelet - Line : Ligne 1, Next : Bastille, Prev : La Defense
Bastille - Line : Ligne 1, Next : terminus, Prev : Chatelet
```

Questions de réflexion à répondre dans le README :

- Quels avantages obtenez-vous avec `shared_ptr` par rapport aux références ?
- Pourquoi cela peut-il créer des problèmes mémoire ? Lesquels ? Utilisez `valgrind` pour mettre en évidence le problème de cycle (par exemple lien bidirectionnel station suivante/précédente ou station/ligne) et ajoutez la trace proposée par `valgrind` dans votre réponse.
- À quel moment les objets sont-ils détruits lorsque l'on utilise des `shared_ptr` ? Comment peut-on vérifier cet instant dans le code (par exemple en ajoutant un message dans le destructeur) ?